

Multi-View Semantic 3D Gaussian Splatting for Component-Aware Bridge Reconstruction from UAV Imagery

Rendering a bridge's appearance and its structural parts from any camera pose, in one pass

Quoc-Bao Ta Trung Ha Nguyen Thai Ha Dang

IC-SHM 2026 — Project 2: Multi-view Semantic 3D Reconstruction of Bridge Structures

[Team identifier]

September 2026

The story in one page

1 The task

Given 400 posed UAV images of a suspension bridge, render both an RGB image and a 5-class semantic map from an arbitrary camera pose.

2 What existed before

3D Gaussian Splatting renders photorealistic RGB but carries no semantics; 2D segmentation gives labels but not a renderable 3D scene.

3 Our backbone

Semantic 3D Gaussian Splatting: give each Gaussian 5 extra semantic logits and render RGB + semantics in one fused rasterization pass.

4 Our additions

Multi-view plurality-vote semantic warm-start; SegFormer pseudo-labels for the 100 unlabeled images; three real bugs found and fixed.

5 The experiment

PSNR 22.43 dB, SSIM 0.854, LPIPS 0.321, structural mIoU 92.08% on 30 held-out test views.

6 What we learned

High cable IoU reflects a coarse annotation convention, not precise cable geometry recovery — and after fixing other bugs, resolution matters less than it first appeared to.

Part 1: The task

- 1 Part 1 — The task**
- 2 Part 2 — Related work
- 3 Part 3 — The backbone
- 4 Part 4 — Our adaptations
- 5 Part 5 — Experiments and results
- 6 Part 6 — Discussion and conclusion

What exactly are we asked to build?

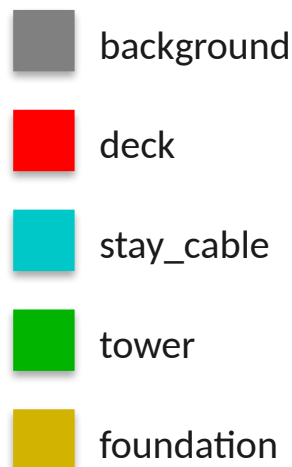
Input

- **400 posed UAV images**
300 carry manual polygon labels; 100 do not
- **Camera calibration + poses**
SfM estimates, documented as “reference only”

Required output

- **RGB image**
matching a held-out photograph
- **Semantic map**
5 classes, see center column
- **...from any query pose**
poses never captured during the flight

5 semantic classes



Think about it: The competition brief calls the physical component “main cable” (p. 9); the dataset’s own class ID is stay_cable. We keep the dataset ID for scoring and name the physical part correctly in prose.

Blind evaluation

- **Visual fidelity**
PSNR, SSIM, LPIPS
- **Semantic accuracy**
mIoU over the 4 structural classes
- **Accuracy Score**
 $= 0.5 \times \text{Visual Fidelity} + 0.5 \times \text{Semantic mIoU}$

Key idea: Our numbers are all measured on a local 30-view holdout carved from the released data — not the organizers’ blind test set.

Why is this hard? Three reasons

1 Coarse cable annotation

Physical cable strands span only a few pixels, but the `stay_cable` polygon encloses the whole cable-and-hanger region — including sky and river between the strands. High overlap with this label does not mean precise strand geometry.

2 A quarter of the images are unlabeled

100 of 400 images carry no manual annotation. Using their viewpoints for semantic supervision requires pseudo-labeling.

3 Camera poses are estimates, not ground truth

Poses come from Structure-from-Motion and are documented as “reference only”, so residual geometric inconsistency can affect rendering quality.

Think about it: None of these three problems is solved by a bigger model — each needs its own targeted fix, covered in Part 4.

How is the answer scored?

$$\text{Accuracy Score} = 0.5 \times \text{Visual Fidelity (PSNR / SSIM / LPIPS)} + 0.5 \times \text{Semantic mIoU}$$

- **PSNR / SSIM**

pixel and structural similarity between rendered and real RGB

- **LPIPS**

a learned perceptual distance (AlexNet backbone), lower is better

- **Structural mIoU**

mean IoU over the 4 structural classes; we also report the 5-class mIoU (including background) for completeness

Think about it: The brief specifies the 0.5/0.5 weighting but does not define how PSNR/SSIM/LPIPS combine into one “Visual Fidelity” number. We document our own combination explicitly as illustrative, not an organizer-defined formula.

Part 2: What existed before

- 1 Part 1 — The task
- 2 Part 2 — Related work**
- 3 Part 3 — The backbone
- 4 Part 4 — Our adaptations
- 5 Part 5 — Experiments and results
- 6 Part 6 — Discussion and conclusion

From implicit fields to explicit, labelable primitives

1 NeRF

Implicit, coordinate-based network; renders via ray-marching. High quality, slow, and no natural place to attach a discrete label.

2 3D Gaussian Splatting (Kerbl et al.)

Explicit anisotropic Gaussians, tile-based rasterization. Real-time, and every pixel traces to an addressable set of primitives — the property we build on.

3 Semantic-NeRF / Feature-3DGS / LangSplat / Gaussian Grouping

Attach extra attributes (semantic logits, distilled 2D features, language embeddings, SAM-derived identity codes) to a renderable representation — but for different supervision and different downstream tasks than ours.

Key idea: Our task needs class IDs for a fixed set of bridge components — so we optimize five class logits directly against real and pseudo-labeled masks, rather than distilling an open-vocabulary feature field.

Where our design sits

Method	Renders RGB	Native semantics	Single fused pass	Fixed class set
NeRF [6]	yes	no	—	—
3D Gaussian Splatting [7]	yes	no	—	—
Semantic-NeRF [8]	yes	yes	no (ray integration)	yes
Feature-3DGS [9] / LangSplat [12]	yes	distilled features	yes	no (open-vocab)
Ours	yes	yes	yes	yes

● Task A (2D pseudo-labeling) uses SegFormer

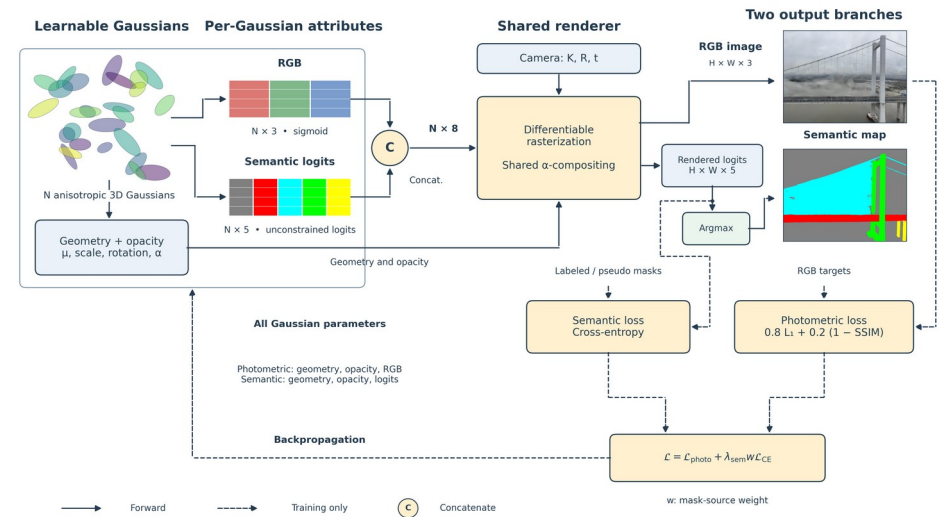
a lightweight hierarchical-Transformer encoder (MiT-B0) with an all-MLP decoder — compact enough to fine-tune on a single GPU

Part 3: The backbone

- 1 Part 1 — The task
- 2 Part 2 — Related work
- 3 Part 3 — The backbone**
- 4 Part 4 — Our adaptations
- 5 Part 5 — Experiments and results
- 6 Part 6 — Discussion and conclusion

What is a 3D Gaussian, intuitively?

- **A soft, blurry ellipsoid in 3D space**
not a point, not a triangle — a smooth splat of color with a position, size, orientation, and opacity
- **A scene = thousands of overlapping splats**
rendering a view means projecting every splat onto the image plane and blending them by depth
- **“Alpha-compositing” = blending front-to-back**
each pixel's color is a weighted sum of every splat that covers it, nearest splats contributing most



Key idea: Because every rendered pixel traces back to an explicit, addressable set of Gaussians, it is natural to give each Gaussian one more attribute beyond color.

Adding a semantic channel: one tensor, one rasterization pass

- **Each Gaussian already carries**

position, scale, rotation, opacity, and RGB color (3 numbers, constrained to [0,1] by a sigmoid)

- **We add 5 more numbers per Gaussian**

unconstrained semantic logits, one per class (background, deck, stay_cable, tower, foundation)

- **Concatenate: $3 + 5 = 8$ channels**

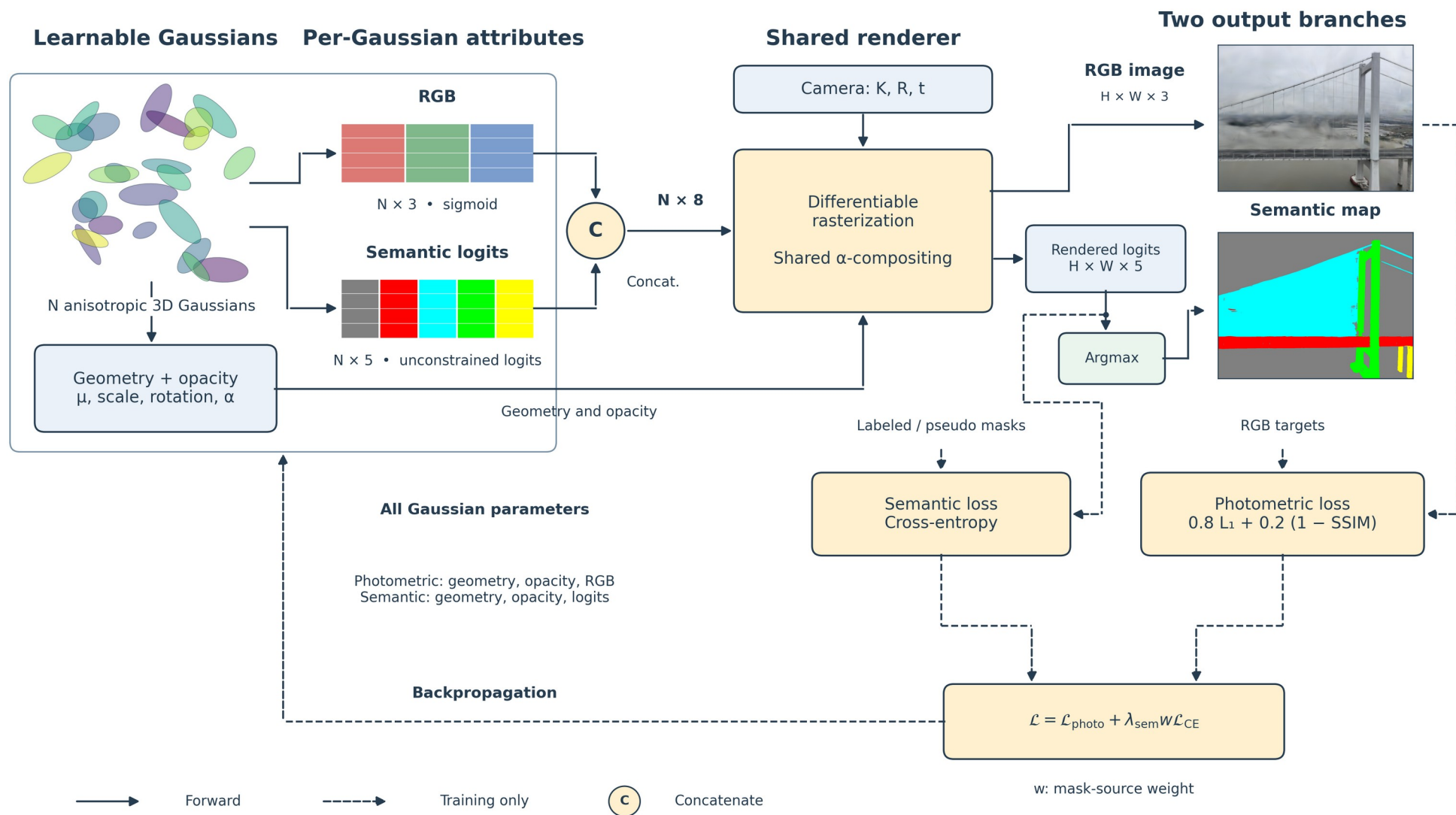
rasterized together in a single gsplat pass, sharing the exact same projection, depth order, and blending weights as color

- **Split the output back apart**

channels 0-2 → RGB image; channels 3-7 → semantic logits → argmax → class map

Key idea: Because RGB and semantics share the same compositing weights, the two outputs are pixel-aligned by construction — no separate alignment step, no extra rasterization cost.

The full picture: representation, rendering, and losses



Part 4: Our adaptations

- 1 Part 1 — The task
- 2 Part 2 — Related work
- 3 Part 3 — The backbone
- 4 Part 4 — Our adaptations**
- 5 Part 5 — Experiments and results
- 6 Part 6 — Discussion and conclusion

Starting from a good guess, not from noise

- **Triangulate a sparse point cloud**

86,319 tracks from the 370 training-pool images (test images excluded, see Part 5) → 84,086 triangulated → 82,518 after outlier filtering

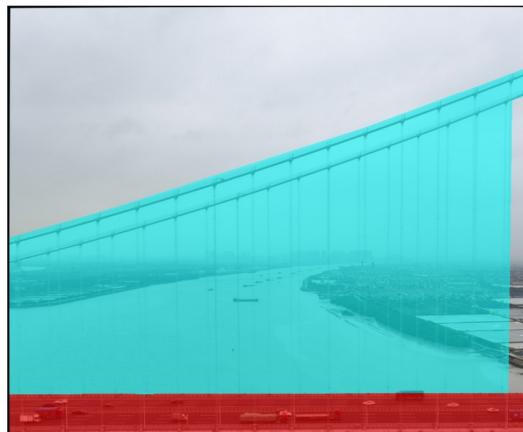
- **Position + color initialize each Gaussian**

not random — real triangulated position, real averaged pixel color from every observing view

- **Semantic logits initialize by multi-view plurality vote**

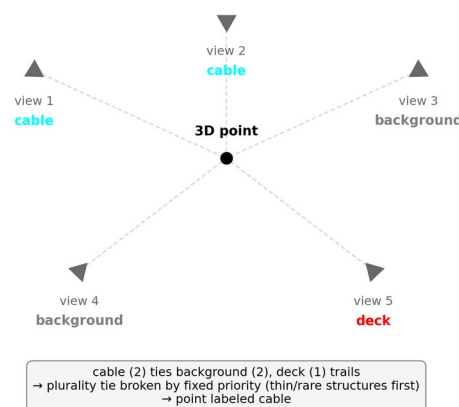
every labeled view observing a point votes for the class at its feature-track pixel; the winning class gets logit +2, others -2 (a confident but not saturated prior)

Real example: GT mask overlaid on an undistorted UAV image

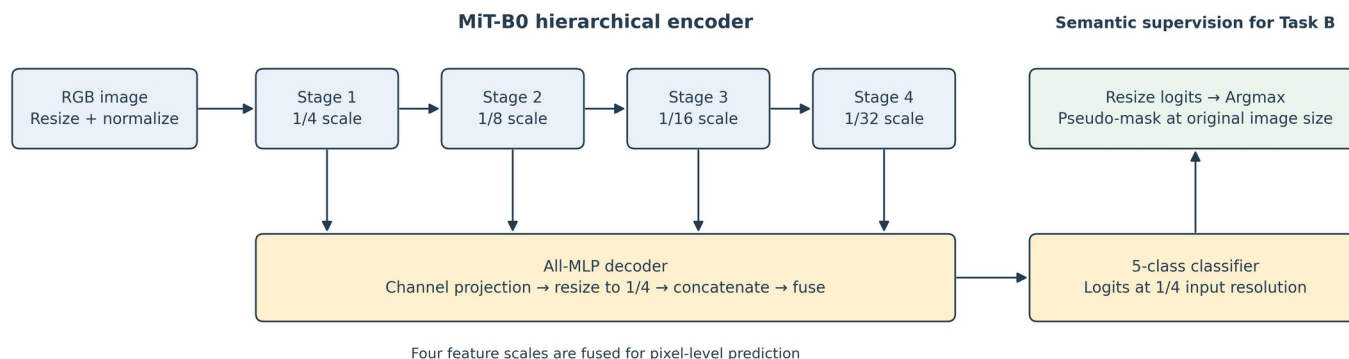


cyan = stay_cable; note the polygon covers sky and river far beyond the cable strands

Illustrative example: multi-view vote for one 3D point



Task A: turning 100 unlabeled images into training signal



- **Fine-tune SegFormer (MiT-B0)**

on 240 labeled images, 80 epochs

- **Select checkpoint by validation mIoU**

on a separate 30-image internal validation split — never the 30-image test split

- **Best validation mIoU: 81.67%**

at epoch 69

- **Predict pseudo-masks for 100 images**

270 labeled + 100 pseudo-labeled = 370 views supervise Task B

Think about it: Checkpoint selection never touches the 30-image test split — an earlier version of our pipeline leaked this and we fixed it (next slides).

Adaptation 1: align masks to the space we actually render in

- Problem:** Ground-truth and pseudo masks are rasterized/predicted on the original, lens-distorted photos. But Gaussian Splatting assumes an ideal pinhole camera, so training and rendering happen in an undistorted image space.
- Consequence:** A mask compared pixel-for-pixel against an undistorted render is silently misaligned by several pixels near the frame edges — worst exactly where thin classes like the cable live.
- Fix:** Undistort every ground-truth and pseudo mask with the same per-pixel remap used for images, with nearest-neighbor interpolation so discrete class IDs never blur.

Key idea: A geometric consistency bug, not a modeling choice — fixed once, at the data layer, so every downstream number is computed on correctly aligned data.

Adaptation 2: initialize Gaussian color from real pixels

Problem: An earlier version of our pipeline initialized each Gaussian's RGB from a fixed 5-color class-legend swatch (the same palette used for figures) — not the point's actual observed appearance.

Hypothesis: Starting the photometric loss from a class-swatch color — not real appearance — makes optimization spend part of its fixed 40,000-step budget just correcting the starting point, converging to a worse final fit.

Fix: Sample each point's real RGB from every observing image, averaged over views.

Metric	Before (class-swatch color)	After (real pixel color)
PSNR	21.83 dB	22.13 dB
Structural mIoU	88.97%	91.04%

Key idea: Test of the hypothesis: every single metric improved after the fix.

Adaptation 3: keep the test split out of the sparse cloud too

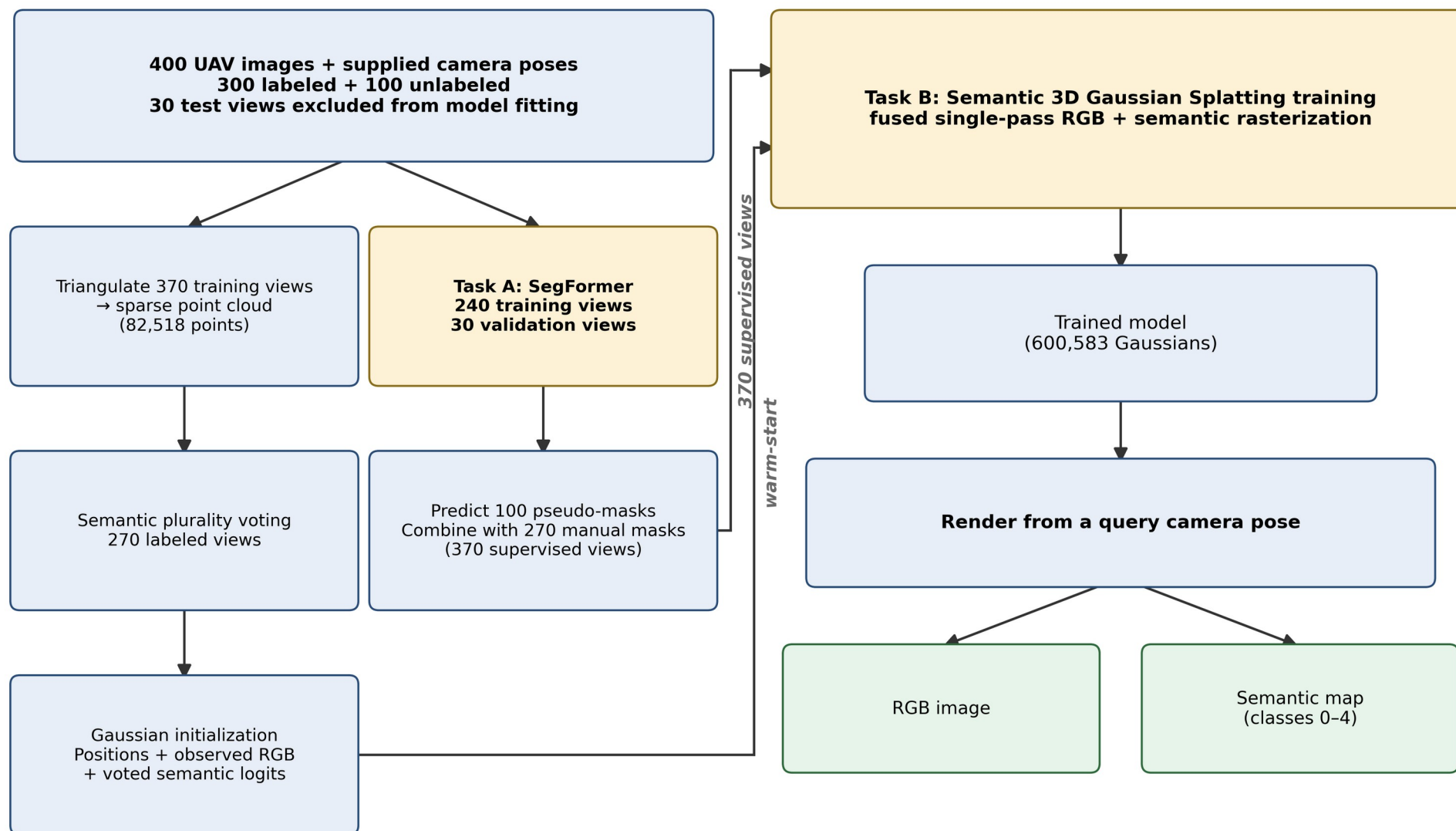
Problem: Our training loss and semantic vote always excluded the 30 test images — but sparse triangulation and Gaussian color sampling used observations from all 400 images, test images included.

Fix: Exclude the 30 test images' 2D observations from every feature track before triangulation; their poses still load (needed to render them at evaluation time), but they cannot influence any point's position or color.

Metric	Before (test leaked in)	After (test fully excluded)
PSNR	22.13 dB	22.43 dB
Structural mIoU	91.04%	92.08%

Key idea: Again, every metric improved — consistent with the dense flight's >99% frame-to-frame overlap making a training-adjacent frame almost as informative as the excluded test frame itself.

The whole pipeline, end to end



Part 5: Experiments and results

- 1 Part 1 — The task
- 2 Part 2 — Related work
- 3 Part 3 — The backbone
- 4 Part 4 — Our adaptations
- 5 Part 5 — Experiments and results**
- 6 Part 6 — Discussion and conclusion

Experimental setup

Data 400 UAV images, 1320×989, shared SIMPLE_RADIAL calibration ($f \approx 925.7$ px, $k_1 \approx 0.009$); 300 labeled, 100 unlabeled

Split 240 / 30 / 30 labeled (train / Task A validation / test), strided by flight order; +100 unlabeled via pseudo-labels

Metrics PSNR, SSIM, LPIPS (visual fidelity); per-class and structural mIoU (semantic accuracy), pooled over all test pixels

Compute Single NVIDIA RTX 3080 (10 GB); 40,000 iterations, seed 42, ≈ 46 min full resolution

Fair comparison Both resolution-ablation runs share the same split, initialization, loss weights, and density-control schedule — only resolution differs

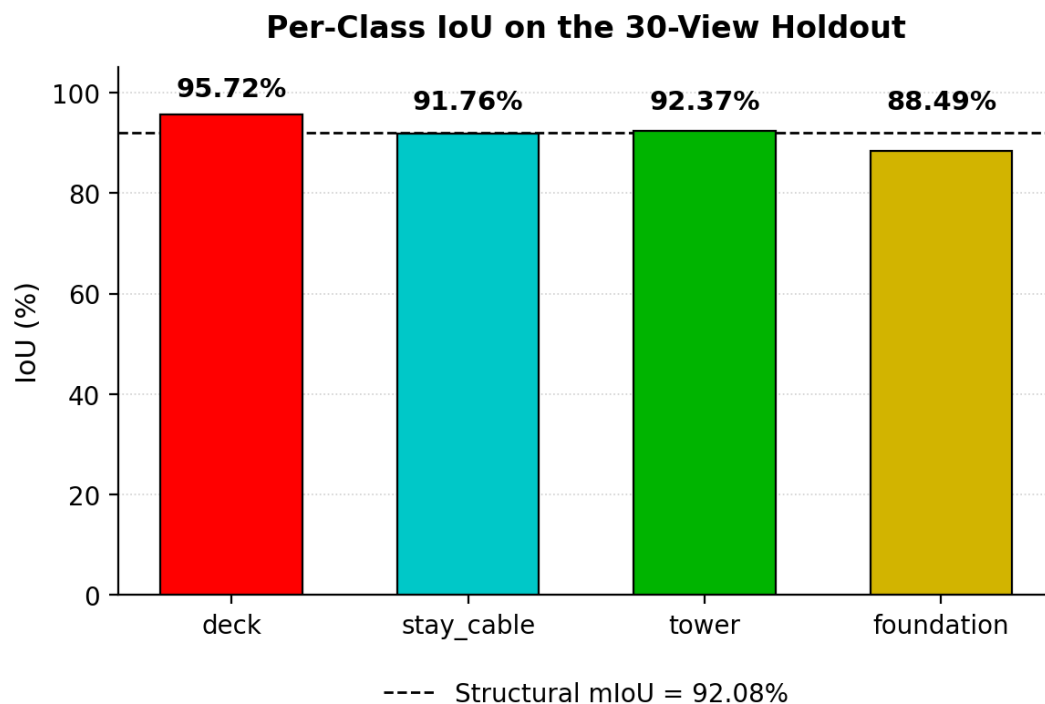
Main result: 30-view local test split

Metric	Value
PSNR	22.43 dB
SSIM	0.854
LPIPS	0.321
Structural mIoU (4 classes)	92.08%
mIoU incl. background (5 classes)	93.5%
Illustrative Accuracy Score	0.823

Class	IoU
deck	95.72%
tower	92.37%
stay_cable	91.76%
foundation	88.49%
background	99.20%

Think about it: For reference: Task A's own 2D validation mIoU is 81.67% (30-image internal validation, not the test split above) — a different task, split, and role.

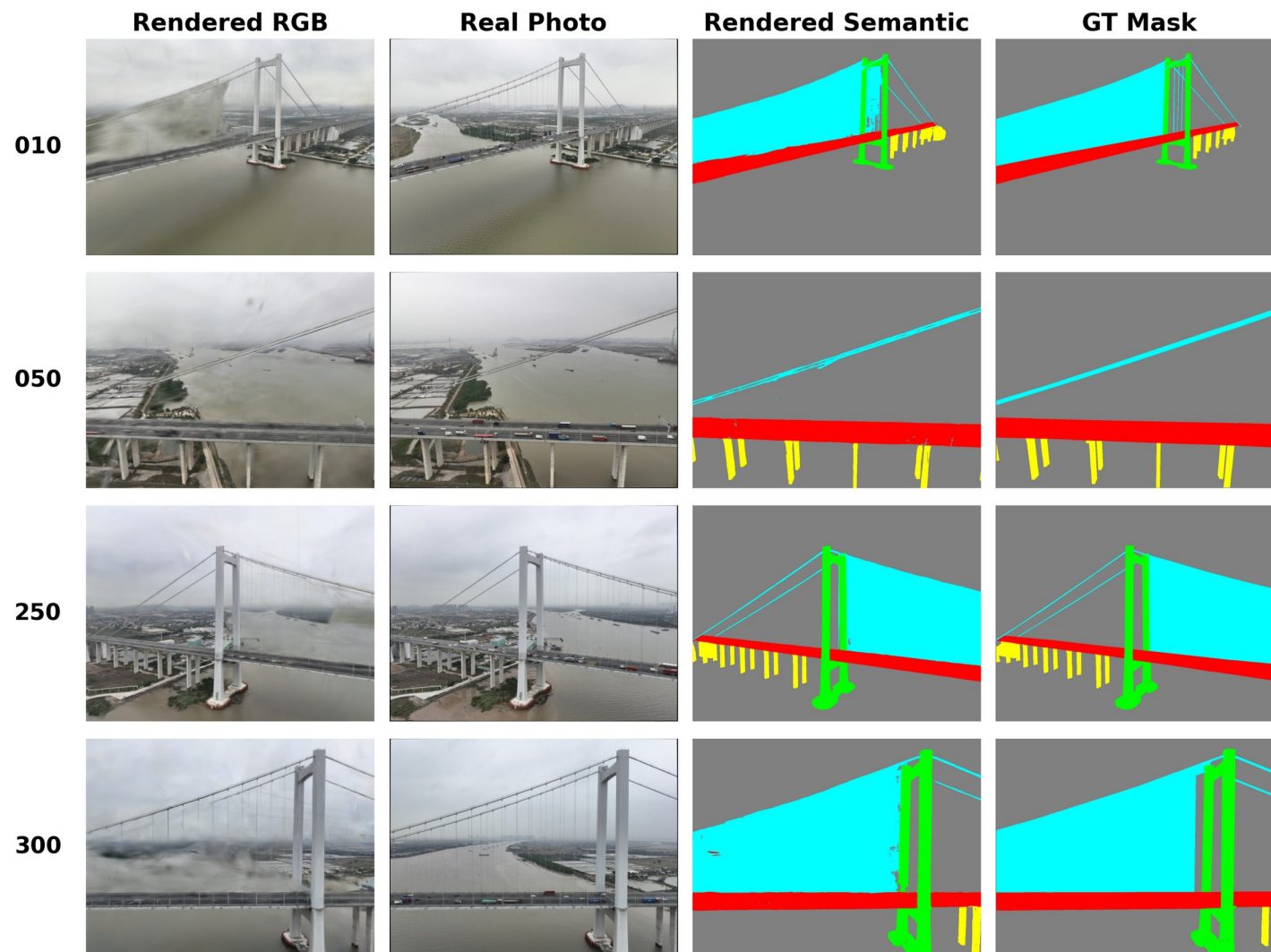
Per-class IoU: why is the thinnest class not the worst?



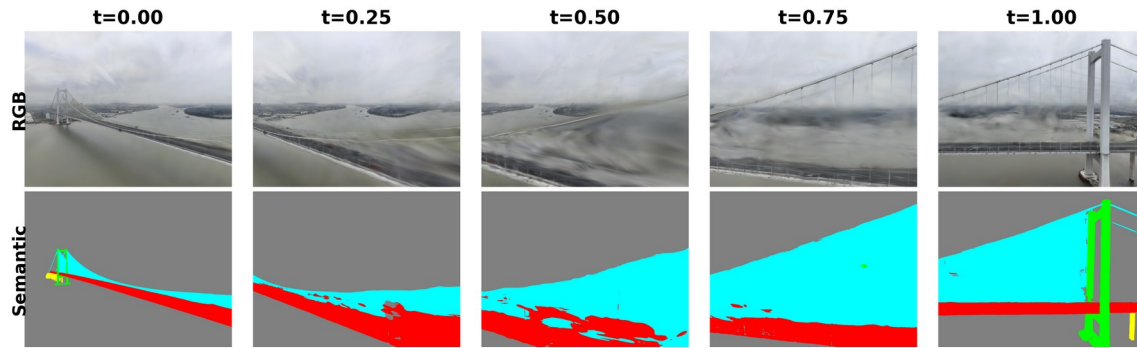
- **Cable strands are physically the thinnest component**
yet stay_cable IoU (91.76%) is within a point of tower and clearly ahead of foundation
- **The scored class is a coarse polygon, not the strands**
cable covers 7.12% of test pixels — more than deck's 4.99% — because the annotation encloses sky and river between strands
- **High cable IoU measures annotation agreement**
not precise recovery of individual cable geometry
- **Foundation's lower IoU is not simply fewer observations**
it receives more average observations per point than deck (6.57 vs. 3.12), so observation count alone does not explain the ranking

Think about it: We report these as descriptive statistics, not a confirmed causal explanation — isolating the true cause would need a dedicated error analysis we have not run.

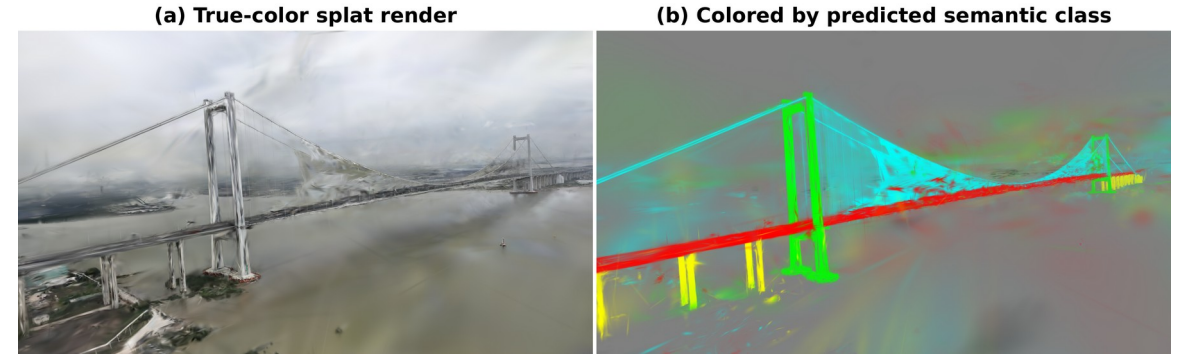
Qualitative check: four held-out views



Beyond the flight line: interpolated poses and the full 3D structure



5 poses interpolated between two real test views (images 280, 300). Endpoints and intermediate frames all show some artifacts; broad component regions stay identifiable but boundaries fragment.



Interactive-viewer screenshots of the same model: (a) learned RGB, (b) each Gaussian recolored by its own argmax class before rendering. Panel (b) is a different visualization from the evaluated metric, which argmaxes after compositing.

Think about it: Rendering a pose is not the same as being accurate at that pose — we show these views to be transparent about where the system still struggles.

Ablation: does training resolution matter?

Training resolution	PSNR	SSIM	LPIPS	Structural mIoU
Half (660×494)	22.03	0.838	0.330	89.71%
Full (1320×989)	22.43	0.854	0.321	92.08%

- **Full resolution wins on every metric**

+0.40 dB PSNR, +0.016 SSIM, -0.009 LPIPS, +2.37 points structural mIoU

- **Cost: ≈46 minutes vs. ≈18 minutes on the same GPU**

a modest absolute difference for the same 40,000-step budget

Think about it: These results support full resolution for this dataset and budget; we have not quantified variation across random seeds, other scenes, or other training budgets.

Part 6: Discussion and conclusion

- 1 Part 1 — The task
- 2 Part 2 — Related work
- 3 Part 3 — The backbone
- 4 Part 4 — Our adaptations
- 5 Part 5 — Experiments and results
- 6 Part 6 — Discussion and conclusion**

Honest limitations

- 1 The split tests interpolation, not extrapolation**
the strided test split places a training frame next to almost every held-out frame along a dense flight path
- 2 Cable IoU reflects an annotation convention**
not verified precision at the level of individual strands
- 3 Accuracy at distant, off-trajectory viewpoints is unverified**
we can render an arbitrary pose, but rendering a pose is not the same as being accurate there
- 4 Not evaluated for the actual SHM end task**
no damage detection, deformation measurement, or structural response estimation has been tested
- 5 Single run per configuration**
no multi-seed variance estimate for the headline numbers or the resolution ablation

Conclusion

- 1 A semantic 3D Gaussian Splatting system that renders aligned RGB and 5-class semantic maps from any query pose in one fused rasterization pass.
- 2 Sparse geometry, observed colors, and multi-view plurality votes warm-start the representation instead of starting from noise.
- 3 SegFormer pseudo-labels extend semantic supervision from 270 to 370 views at no extra annotation cost.
- 4 On a local 30-view test split: PSNR 22.43 dB, SSIM 0.854, LPIPS 0.321, 92.08% structural mIoU.
- 5 Three real pipeline bugs were found and fixed during development — mask alignment, color initialization, and test-split leakage into triangulation — each verified by a measured before/after improvement.
- 6 Results establish agreement with the released annotations near the flight trajectory, not survey-grade geometric accuracy or unrestricted novel-view fidelity.

References and code

- Kerbl, Kopanas, Leimkühler, Drettakis (2023) — 3D Gaussian Splatting for Real-Time Radiance Field Rendering. ACM ToG.
- Mildenhall et al. (2020) — NeRF: Representing Scenes as Neural Radiance Fields. ECCV.
- Zhi, Laidlow, Leutenegger, Davison (2021) — Semantic-NeRF. ICCV.
- Zhou et al. (2024) — Feature 3DGS. CVPR. • Qin et al. (2024) — LangSplat. CVPR.
- Xie et al. (2021) — SegFormer. NeurIPS. • Ye et al. (2025) — gsplat. JMLR.
- Lin, Abe, Zheng, Li, Chun (2025) — A structure-oriented loss function for bridge point clouds. CACAIE.
- Full reference list: [paper/DRAFT.md](#) References (19 numbered entries).

Code: github.com/zeki-aitech/ic-shm-2026-project-2

Thank you — questions?